CSS3122 DATA STRUCTURE AND ALGORITHMS

CH2 ALGORITHMS & PSEUDOCODE

KHAIRUNNISA IBR

CHAPTER OUTLINE

In this chapter, you will learn about:

1. Algorithm definition
2. Characteristics of algorithm
3. Elements of algorithm
4. Pseudocode example
5. Differences of algorithm and pseudocode
6. Algorithms analysis: Big-O Notation
7. Experimental and theoretical analysis

ALGORITHM DEFINITION

DEFINITION

Algorithm → A formally defined procedure for performing some calculations

- Provides **a blueprint to write a program** to solve a particular problem.
- Considered to be **an effective procedure** to solve a problem in a **finite number of steps**.
- **A well-defined algorithm:**
  - **always provides an answer.**
  - **guaranteed to terminate.**

ALGORITHM DEFINITION (CONT)

ALGORITHM

- are mainly used **to achieve software re-use**.
- If a procedure is formally defined, it can then be implemented using any formal language, i.e. the programming language.
  - E.g. C
  - C++
  - Java
  - Etc
- It is not uncommon to have multiple algorithms to tackle the same problem.

CHARACTERISTICS OF ALGORITHM

- Not all procedures can be called an algorithm.
- An algorithm should have the following characteristics:

Unambiguous	Input	Output
Finiteness	Feasibility	Independent

CHARACTERISTICS OF ALGORITHM (CONT)

Unambiguous	• Algorithm should be clear and unambiguous. Each of its steps (or phases), and their inputs/outputs should be clear and must lead to only one meaning.
Input	• An algorithm may have many inputs or no inputs at all.
Output	• An algorithm should have 1 or more well-defined outputs, and should match the desired output.
Finiteness	• Algorithms must terminate after a finite number of steps.
Feasibility	• Should be feasible with the available resources.
Independent	• An algorithm should have step-by-step directions, which should be independent of any programming code.

ELEMENTS OF ALGORITHM

- The following guidelines must be followed while developing an algorithms:

1. An algorithm will be enclosed by **START** (or **BEGIN**) and **STOP** (or **END**).
2. To accept data from user, generally used statements are **INPUT**, **READ**, **GET** or **OBTAIN**.
3. To display result or any message, generally used statements are **PRINT**, **DISPLAY**, or **WRITE**.
4. Generally, **COMPUTE** or **CALCULATE** is used while describing mathematical expressions and based on situation relevant operators can be used.

ELEMENTS OF ALGORITHM (CONT)

- An algorithm has a finite number of steps.
- Some steps may involve decision-making and repetition.
- An algorithm may consist of the following structures:
    1. Sequence.
    2. Decision.
    3. Repetition.

ELEMENTS OF ALGORITHM (CONT)

1. Sequence
   - Each step of an algorithm is executed in a specified order.
   - E.g. An algorithm to add 2 numbers is performed in a sequential order:

```
Step 1: START
Step 2: INPUT first number as A
Step 3: INPUT second number as B
Step 4: SET SUM = A+B
Step 5: PRINT SUM
Step 6: END
```

ELEMENTS OF ALGORITHM (CONT)

2. Decision
   - Used when the execution of a process depends on the outcome of some condition.
   - It can be stated in the following manner:
     - **IF condition Then process** (← an if statement)
     - **IF condition** (← an if else statement)

       Then process1

       ELSE process2
   - E.g. algorithm to test for equality of 2 numbers:

```
Step 1:   START
Step 2:   INPUT first number as A
Step 3:   IF A = B
              PRINT "EQUAL"
          ELSE
              PRINT "NOT EQUAL"
          [END IF]
Step 4:   END
```

ELEMENTS OF ALGORITHM (CONT)

3. Repetition
   - Executing one or more steps for a number of times.
   - Can be implemented using constructs such as while, do-while, and for loops.
   - Loops are executed until some condition is TRUE.
   - E.g. algorithm that prints the first 10 natural numbers:

```
Step 1:   START
Step 2:   [INITIALIZE] SET I = 1, N = 10
Step 3:   Repeat Steps 4 and 5 while I<=N
Step 4:   PRINT I
Step 5:   SET I = I+1
          [END OF LOOP]
Step 6:   END
```

PSEUDOCODE EXAMPLE

- Pseudocode is the expressive way to describe an algorithm.
- A combination between natural language and high-level programming practices that represent the general concept behind an algorithm.
- E.g. Procedure to swap A with B which are obtained from user:

ALGORITHM

Step 1: START
Step 2: INPUT first number as A
Step 3: INPUT second number as B
Step 4: SET TEMP = A
Step 5: SET A = B
Step 6: SET B = TEMP
Step 7: PRINT A, B
Step 8: END

PSEUDOCODE

- User input first number
- User input second number
- Store A in TEMP
- Set A = B
- Set B = TEMP
- Print values of A and B

PSEUDOCODE EXAMPLE (CONT)

- E.g. Procedure to determine whether a student pass or fail based on his average marks.

ALGORITHM

```
Step 1:   START
Step 2:   INPUT M1, M2, M3
Step 3:   GRADE ← (M1+M2+M3)/3
Step 4:   IF (GRADE < 50)
              PRINT "FAIL"
          ELSE
              PRINT "PASS"
          [ENDIF]
Step 5:   END
```

PSEUDOCODE

- User input 3 marks
- Store average of marks in GRADE
- If GRADE is less than 50
- Print "FAIL"
- Else
- Print "PASS"

DIFFERENCES OF ALGORITHM AND PSEUDOCODE

ALGORITHM	PSEUDOCODE
Provides the procedure **how the program will be carried** out to get the expected output	Provides an **outline of the general flow** of the program
Use **complicated programming language**	Use combination of **programming language & natural language**
Complex to be written	**Easier** to be written
Offers solution for the problem	**Offers explanation** of the algorithm
Helpful in **understanding the execution** of the program	Helpful in **understanding the idea** of the program

ALGORITHMS ANALYSIS: BIG-O NOTATION

- The number of statements executed in a program for n elements of data is a function of the number of elements, expressed as f(n).

E.g:

```
for (i=1; i<10; i++)
        for (j=0; j<i; j++)
                statement;
```

The inner loop in this code execute just once in the 1st iteration, twice in the 2nd iteration, thrice in the 3rd iteration, and so on.
Hence, the no of iteration is:

$$1 + 2 + 3 + \ldots + 9 = 45$$

The average of this loop can be calculated as:

$$= 45/9 = 5$$

$$= \text{No of outer loop's iteration} + 1 \;/\; 2$$

$$= (9+1)/2$$

The inner loop iterates (n + 1)/2 times.
Hence, the efficiency of the code can be given as f(n) = n(n+1)/2.

ALGORITHMS ANALYSIS: BIG-O NOTATION (CONT)

- In today's era, we are hardly concerned with the efficiency of algorithm.
- But, we are more concern with the generic order of the magnitude of the algorithm.

BIG-O Notation

- O stands for '**order of**'.
- It is concerned with **what happens for every large values of n**.
- E.g. the statements executed in a program can be represented in the form of **a function of n number of elements**, **f(n)**.
  - A dominant factor in the complex function expression is sufficient to determine the order of the magnitude of the algorithm, hence the efficiency of the algorithm.
  - This factor is the **Big O** (expressed as **O(n)**).

ALGORITHMS ANALYSIS: BIG-O NOTATION (CONT)

- Example:
  - If a sorting algorithm performs **n^2 operations to sort just n elements**, the algorithm is **an $O(n^2)$ algorithm.**
  - More examples:

Execution of algorithm in function expression	Big O Notation
$f(n) = 10$	$O(1)$
$f(n) = 2n^3 + 1$	$O(n^3)$
$f(n) = 3n^2 + 5$	$O(n^2)$
$f(n) = 2n^3 + 3n^2 + 5n - 10$	$O(n^3)$

ALGORITHMS ANALYSIS: BIG-O NOTATION (CONT)

- According to the Big O notation, we have 5 different categories of algorithms:

Constant linear algorithm	• running time complexity given as O(1).
Linear time algorithm	• Running time complexity given as O(n).
Logarithmic time algorithm	• Running time complexity given as O(log n).
Polynomial time algorithm	• Running time complexity given as $O(n^k)$ where k>1.
Exponential time algorithm	• Running time complexity given as $O(e^n)$.

ALGORITHMS ANALYSIS: BIG-O NOTATION (CONT)

- The number of operations that would be performed for various values of n are as follows:

n	O(1)	O(log n)	O(n)	O(n log n)	$O(n^2)$	$O(n^3)$
1	1	1	1	1	1	1
2	1	1	2	2	4	8
4	1	2	4	8	16	64
8	1	3	8	24	64	512
16	1	4	16	64	256	4096

EXPERIMENTAL AND THEORETICAL ANALYSIS

- Analysing an algorithm = determining the number of resources (e.g. time and memory) needed to execute it.
- **Efficiency or complexity of an algorithm** is stated in terms of **time and space complexity:**
  1. **Space complexity** – amount of computer memory required during the program execution as a function of the input size.
  2. **Time complexity** – the running time of a program as a function of the input size.

EXPERIMENTAL AND THEORETICAL ANALYSIS

1. Space complexity

- Space needed by a program is divided into 2 parts:

Fixed part	• Includes the space needed to store instructions, constant, variables and structured variables (e.g. array, structures).
Variable part	• Includes the space needed for recursion stack, and for structured variables that are allocated dynamically during the running of a program.

EXPERIMENTAL AND THEORETICAL ANALYSIS

2. Time complexity

- Time complexity of a program is divided into 3 types:

Worst case running time	• Used to analyse behaviour of an algorithm with respect to the **worse possible case of input**. • If we know this, we know the running time of the algorithm will not go beyond this limit. • e.g. worst-case for a sorting algorithm (to sort in increasing order) is when all elements in the array is arranged in a descending order.
Average-case running time	• Used to analyse behaviour of algorithm **when input is randomly drawn from a distribution**. • Estimate of running time for average input.
Best-case running time	• Used to analyse an algorithm under **optimal condition**. • e.g. best case for a linear search on an array occurs when the desired elements is the first array element.